

Дизайн и анализ на алгоритми упражнение 13

Станислав Игов (Стъката)

2025/2026

Динамично програмиране - продължение. 0/1 Раница. Верижно матрично умножение. Разстояние на Левенщайн.

Преговор

Нека си припомним петте стъпки за решаване на ДП задача:

1. **Дефинирай подзадачите.** Какво точно пресмята $dp[i]$ (или $dp[i][j]$)?
2. **Намери рекурентната връзка.** Как $dp[i]$ зависи от по-малки подзадачи?
3. **Определи базовите случаи.** Кои подзадачи знаем директно?
4. **Определи реда на пресмятане.** В какъв ред попълваме таблицата?
5. **Определи крайния отговор.** Коя клетка (или комбинация) дава отговора?

При всяка задача ключовият въпрос е: *кое е последното решение, взето в оптималния отговор?* Отговорът на този въпрос определя рекурентната връзка.

Задача 1: 0/1 Раница (0/1 Knapsack)

Условие: Дадени са n предмета. Предмет i има тегло $w[i]$ и стойност $v[i]$. Имаме раница с капацитет W . Искаме да изберем подмножество от предмети с максимална обща стойност, без да надвишаваме капацитета. Всеки предмет може да се вземе **най-много веднъж** (0 или 1 пъти) - оттук идва наименованието "0/1".

Стъпка 1: Дефиниция на подзадачата

Нека:

$dp[i][j]$ = максималната стойност, постижима с предмети $\{1, \dots, i\}$ и капацитет j

Стъпка 2: Рекурентна връзка

За всеки предмет i имаме точно **два избора**:

- **Не вземаме предмет i :** оптималната стойност идва изцяло от предметите $\{1, \dots, i-1\}$ с капацитет j .
- **Вземаме предмет i** (само ако $w[i] \leq j$): "изразходваме" $w[i]$ капацитет и търсим оптимума за $\{1, \dots, i-1\}$ с намален капацитет $j - w[i]$.

Вземаме по-добрия от двата варианта:

$$dp[i][j] = \begin{cases} dp[i-1][j] & \text{ако } w[i] > j \\ \max(dp[i-1][j], dp[i-1][j - w[i]] + v[i]) & \text{ако } w[i] \leq j \end{cases}$$

Стъпка 3: Базови случаи

$dp[0][j] = 0$ за всяко j (няма предмети - нулева стойност)

$dp[i][0] = 0$ за всяко i (нулев капацитет - нулева стойност)

Стъпка 4: Ред на пресмятане и псевдокод

Погълваме таблицата ред по ред: $i = 1, 2, \dots, n$, а за всяко $i - j = 0, 1, \dots, W$.

Knapsack($W, w[1..n], v[1..n]$):

```
@1  dp[0..n][0..W] <- malloc((n+1) * (W+1)){0}
@2  for i from 1 to n:
@3      for j from 0 to W:
@4          if w[i] > j:
@5              dp[i][j] <- dp[i-1][j]                // случай 1: не можем да вземем i
@6          else:
@7              dp[i][j] <- max( dp[i-1][j],           // случай 2a: не вземаме i
@8                              dp[i-1][j - w[i]] + v[i] ) // случай 2b: вземаме i
@9          endif
@10     endfor
@11 endfor
@12 return dp[n][W]
```

Отговор: $dp[n][W]$.

Сложност: $O(n \cdot W)$ по време и памет.

Защо рекурентната връзка е коректна?

Ключовото наблюдение е, че оптималното решение *или* съдържа предмет i , *или* не. Тези два случая са **изчерпателни и взаимно изключващи се**. Следователно оптимумът е максимумът от двата сценария.

Подзадачите $dp[i-1][\cdot]$ са строго по-малки (индексът i намалява), така че не се получават циклични зависимости. Всяка подзадача се изчислява точно веднъж - оттук произтича ефективността.

Коректност (индукция по i)

Твърдение: $dp[i][j]$ е максималната стойност, постижима с предмети $\{1, \dots, i\}$ и капацитет j .

База ($i = 0$): Нямаме предмети, следователно единствената постижима стойност е 0. $dp[0][j] = 0$.

Индукционна стъпка: Предполагаме, че $dp[i-1][\cdot]$ е коректно пресметнато. Нека OPT е оптималното подмножество за предмети $\{1, \dots, i\}$ с капацитет j .

- **Случай 1: Предмет $i \notin OPT$.** Тогава OPT е оптимално подмножество на $\{1, \dots, i-1\}$ с капацитет j . По индукционна хипотеза стойността му е $dp[i-1][j]$. Формулата взема $\max(dp[i-1][j], \dots) \geq dp[i-1][j]$.
- **Случай 2: Предмет $i \in OPT$.** Тогава $OPT \setminus \{i\}$ е оптимално подмножество на $\{1, \dots, i-1\}$ с капацитет $j - w[i]$. (Ако имаше по-добро, можехме да добавим обратно предмет i и да подобрим OPT - противоречие.) По индукционна хипотеза стойността на $OPT \setminus \{i\}$ е $dp[i-1][j - w[i]]$, следователно стойността на OPT е $dp[i-1][j - w[i]] + v[i]$.

И в двата случая формулата изчислява точно стойността на OPT .

Пример

Нека $n = 3$, $W = 5$, с данни:

Предмет i	Тегло $w[i]$	Стойност $v[i]$
1	2	6
2	2	10
3	3	12

	$j = 0$	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$
$i = 0$	0	0	0	0	0	0
$i = 1$	0	0	6	6	6	6
$i = 2$	0	0	10	10	16	16
$i = 3$	0	0	10	12	16	22

Отговор: $dp[3][5] = 22$ - вземаме предмети 2 и 3 (тегло $2 + 3 = 5$, стойност $10 + 12 = 22$).

Задача 2: Верижно матрично умножение

Условие: Дадена е поредица от матрици $A_1, A_2, \dots, A_n$, където A_i има размерност $p[i-1] \times p[i]$. Матричното умножение е асоциативно, следователно можем да поставяме скобите по различни начини. Намерете наредба на скобите, която **минимизира броя скаларни умножения**.

Припомняне: Умножаването на матрица $(a \times b)$ по матрица $(b \times c)$ изисква $a \cdot b \cdot c$ скаларни умножения.

Мотивация

Нека имаме три матрици $A_1 : 10 \times 30$, $A_2 : 30 \times 5$, $A_3 : 5 \times 60$.

- **Вариант 1:** $(A_1 \cdot A_2) \cdot A_3$ — първо умножаваме A_1 и A_2 :

- $A_1 \cdot A_2$: $10 \cdot 30 \cdot 5 = 1500$ умножения, резултат 10×5
- $(A_1 A_2) \cdot A_3$: $10 \cdot 5 \cdot 60 = 3000$ умножения
- **Общо: 4500**

- **Вариант 2:** $A_1 \cdot (A_2 \cdot A_3)$ — първо умножаваме A_2 и A_3 :

- $A_2 \cdot A_3$: $30 \cdot 5 \cdot 60 = 9000$ умножения, резултат 30×60
- $A_1 \cdot (A_2 A_3)$: $10 \cdot 30 \cdot 60 = 18000$ умножения
- **Общо: 27000**

Същият математически резултат, но **6 пъти разлика** в броя операции!

Наблюдение: Оптималното нареждане на скобите задължително прави някакво *последно умножение* - то разделя цялата верига на ляво и дясно парче. Но **на коя позиция k се поставя това последно разделяне?** Не знаем предварително, затова изпробваме всички възможни позиции и вземаме най-добрата.

Стъпка 1: Дефиниция на подзадачата

Нека:

$$dp[i][j] = \text{минималният брой скаларни умножения за пресмятане на } A_i \cdot A_{i+1} \cdots A_j$$

Стъпка 2: Рекурентна връзка

За да изчислим $A_i \cdots A_j$, трябва на някоя позиция k (където $i \leq k < j$) да “разрежем” веригата:

$$(A_i \cdots A_k) \cdot (A_{k+1} \cdots A_j)$$

- Лявата група дава матрица с размерност $p[i-1] \times p[k]$ и струва $dp[i][k]$.
- Дясната група дава матрица с размерност $p[k] \times p[j]$ и струва $dp[k+1][j]$.
- Умножаването им струва $p[i-1] \cdot p[k] \cdot p[j]$.

Минимизираме по всички възможни k :

$$dp[i][j] = \min_{i \leq k < j} \left(dp[i][k] + dp[k+1][j] + p[i-1] \cdot p[k] \cdot p[j] \right)$$

Стъпка 3: Базов случай

$$dp[i][i] = 0 \quad \text{за всяко } i \quad (\text{единична матрица - не се умножава})$$

Стъпка 4: Ред на пресмятане и псевдокод

Пресмятаме по нарастваща дължина на интервала: $\ell = 2, 3, \dots, n$, като за всяка дължина итерираме по всички начални точки i .

```
MatrixChain(n, p[0..n]):
@1  dp[1..n][1..n] <- malloc(n*n){0}
@2  for length from 2 to n:
@3      for i from 1 to n - length + 1:
@4          j <- i + length - 1
@5          dp[i][j] <- infinity
@6          for k from i to j - 1:           // пробваме всяка позиция на последния разрез
@7              cost <- dp[i][k]             // оптимална цена на лявото парче
@8                  + dp[k+1][j]             // оптимална цена на дясното парче
@9                  + p[i-1]*p[k]*p[j]       // цена на последното умножение
@10             if cost < dp[i][j]:
@11                 dp[i][j] <- cost
@12             endif
@13         endfor
@14     endfor
@15 endfor
@16 return dp[1][n]
```

Отговор: $dp[1][n]$.

Сложност: $O(n^3)$ по време, $O(n^2)$ по памет.

Защо рекурентната връзка е коректна?

Всяко поставяне на скоби определя **двоично дърво** на умножения. Коренът на дървото съответства на последното умножение. “Разрязването” на позиция k определя точно последното умножение: лявото поддърво умножава $A_i \cdots A_k$, а дясното - $A_{k+1} \cdots A_j$.

Оптималното решение задължително прави последно умножение на *някоя* позиция k . За всяка такава позиция лявото и дясното поддърво трябва да са оптимални поотделно (оптимална подструктура). Следователно е достатъчно да пробваме всички k и да вземем минимума.

Коректност (индукция по дължина на интервала)

Твърдение: $dp[i][j]$ е минималният брой скаларни умножения за $A_i \cdots A_j$.

База ($j = i$): Произведение от една матрица не изисква умножения. $dp[i][i] = 0$.

Индукционна стъпка: Нека $\ell = j - i \geq 1$. Предполагаме, че $dp[i'][j']$ е коректно за всички интервали с дължина $< \ell$. Нека OPT е оптималното наредение за $A_i \cdots A_j$ и нека k^* е позицията на последното умножение в OPT . Тогава:

$$|OPT| = dp[i][k^*] + dp[k^* + 1][j] + p[i - 1] \cdot p[k^*] \cdot p[j]$$

(по индукционна хипотеза, тъй като двата подинтервала имат дължини $< \ell$). Тъй като формулата взема минимум по всички k , тя е $\leq |OPT|$. Но тя не може да даде по-малко от оптимума, следователно $dp[i][j] = |OPT|$.

Пример

Нека $n = 4$ матрици с размерности: $A_1 : 10 \times 30$, $A_2 : 30 \times 5$, $A_3 : 5 \times 60$, $A_4 : 60 \times 10$.

Масивът с размерностите е $p = [10, 30, 5, 60, 10]$.

	$j = 1$	$j = 2$	$j = 3$	$j = 4$
$i = 1$	0	1500	7500	8000
$i = 2$	–	0	9000	4500
$i = 3$	–	–	0	30000
$i = 4$	–	–	–	0

Отговор: $dp[1][4] = 8000$. Оптималното наредение е $((A_1 \cdot A_2) \cdot (A_3 \cdot A_4))$, т.е. разрезът е на $k = 2$.

За сравнение: наивното ляво-асоциативно умножение $((A_1 \cdot A_2) \cdot A_3) \cdot A_4$ изисква $1500 + 10 \cdot 5 \cdot 60 + 10 \cdot 60 \cdot 10 = 1500 + 3000 + 6000 = 10500$ умножения - с 31% повече!

Задача 3: Разстояние на Левенщайн (Edit Distance)

Условие: Дадени са две думи $A = a_1 a_2 \dots a_m$ и $B = b_1 b_2 \dots b_n$. Позволен са три операции, всяка с цена 1:

- **Вмъкване** на символ c на позиция i : $a_1 \dots a_{i-1} a_i \dots a_m \rightarrow a_1 \dots a_{i-1} c a_i \dots a_m$
- **Изтриване** на символ на позиция i : $a_1 \dots a_{i-1} a_i a_{i+1} \dots a_m \rightarrow a_1 \dots a_{i-1} a_{i+1} \dots a_m$
- **Замяна** на символ на позиция i с символ c : $a_1 \dots a_{i-1} a_i a_{i+1} \dots a_m \rightarrow a_1 \dots a_{i-1} c a_{i+1} \dots a_m$

Намерете минималния брой операции за трансформиране на A в B .

Мотивация

Нека трансформираме думата **kot** в **kost**. Ето три различни поредици от операции:

- **Вариант 1** (1 операция): **kot** $\xrightarrow{\text{вмъкни s на поз. 3}}$ **kost**
- **Вариант 2** (3 операции): **kot** $\xrightarrow{\text{замени o} \rightarrow \text{s}}$ **kst** $\xrightarrow{\text{вмъкни o на поз. 2}}$ **kost** — стигаме до същия резултат, но с повече операции.
- **Вариант 3** (2 операции): **kot** $\xrightarrow{\text{замени t} \rightarrow \text{s}}$ **kos** $\xrightarrow{\text{вмъкни t на поз. 4}}$ **kost**

Тук лесно виждаме, че Вариант 1 е оптимален. Но при по-дълги думи ситуацията бързо става неочевидна — например за **kitten** $\rightarrow$ **sitting** трябва да разгледаме огромен брой възможни поредици от операции. Именно затова използваме ДП: вместо да изброяваме всички поредици, свеждаме задачата до подзадачи за по-къси префикси.

Стъпка 1: Дефиниция на подзадачата

Нека:

$$dp[i][j] = \text{минималният брой операции за трансформиране на } A[1..i] \text{ в } B[1..j]$$

Стъпка 2: Рекурентна връзка

Ключовият въпрос: Каква е последната операция в оптималната трансформация на $A[1..i]$ в $B[1..j]$? Тя задължително е една от трите — и всеки случай свежда задачата до строго по-малък префикс:

- **Съвпадение или замяна на позиция (i, j) :** Последната операция съпоставя $a[i]$ с $b[j]$. Преди нея сме трансформирали $A[1..i-1]$ в $B[1..j-1]$.

$$\text{cost} = dp[i-1][j-1] + [a[i] \neq b[j]]$$

- **Изтриване на $a[i]$:** Последната операция изтрива $a[i]$. Преди нея сме трансформирали $A[1..i-1]$ в $B[1..j]$.

$$\text{cost} = dp[i-1][j] + 1$$

- **Вмъкване на $b[j]$:** Последната операция вмъква $b[j]$ в края. Преди нея сме трансформирали $A[1..i]$ в $B[1..j-1]$.

$$\text{cost} = dp[i][j-1] + 1$$

Вземаме минимума от трите случая:

$$dp[i][j] = \min\left(dp[i-1][j-1] + [a[i] \neq b[j]], dp[i-1][j] + 1, dp[i][j-1] + 1\right)$$

Стъпка 3: Базови случаи

$dp[0][j] = j$ (вмъкваме j символа за да получим $B[1..j]$ от празен низ)
 $dp[i][0] = i$ (изтриваме i символа за да получим празен низ от $A[1..i]$)

Стъпка 4: Ред на пресмятане и псевдокод

Попълваме таблицата ред по ред: $i = 1, 2, \dots, m$, за всяко $i - j = 1, 2, \dots, n$.

```
EditDistance(A[1..m], B[1..n]):
@1  dp[0..m][0..n] <- malloc((m+1)*(n+1))
@2  for i from 0 to m: dp[i][0] <- i          // база: изтриваме i символа
@3  for j from 0 to n: dp[0][j] <- j          // база: вмъкваме j символа
@4  for i from 1 to m:
@5      for j from 1 to n:
@6          if A[i] = B[j]:
@7              dp[i][j] <- dp[i-1][j-1]      // съвпадение, цена 0
@8          else:
@9              dp[i][j] <- min(
@10                 dp[i-1][j-1] + 1,          // замяна на a[i] с b[j]
@11                 dp[i-1][j] + 1,             // изтриване на a[i]
@12                 dp[i][j-1] + 1 )            // вмъкване на b[j]
@13          endif
@14      endfor
@15 endfor
@16 return dp[m][n]
```

Отговор: $dp[m][n]$.

Сложност: $O(m \cdot n)$ по време и памет.

Защо рекурентната връзка е коректна?

Нека си представим трансформацията като **изравняване** (alignment) на двата низа. Всяка позиция в изравняването отговаря на един от трите случая: съвпадение/замяна, изтриване или вмъкване.

Ако оптималната трансформация завършва с операция, свързана с $a[i]$ и/или $b[j]$, тя свежда задачата до по-кратък префикс. Подзадачите са строго по-малки (сумата $i + j$ намалява), следователно няма циклични зависимости. Трите случая са **изчерпателни** — всяка трансформация задължително завършва с един от тях.

Коректност (индукция по $i + j$)

Твърдение: $dp[i][j]$ е минималното разстояние между $A[1..i]$ и $B[1..j]$.

База ($i = 0$): Единственият начин да трансформираме празния низ в $B[1..j]$ е с точно j вмъквания. $dp[0][j] = j$. Симетрично $dp[i][0] = i$.

Индукционна стъпка: Нека $i, j \geq 1$. Предполагаме, че $dp[i'][j']$ е коректно за всички двойки с $i' + j' < i + j$. Нека OPT е оптималната трансформация на $A[1..i]$ в $B[1..j]$. Разглеждаме последната операция в OPT :

- **Замяна/съвпадение на (i, j) :** Преди нея $A[1..i-1]$ е трансформирано в $B[1..j-1]$ оптимално. По индукционна хипотеза цената е $dp[i-1][j-1]$. Следователно $|OPT| = dp[i-1][j-1] + [a[i] \neq b[j]]$.
- **Изтриване на $a[i]$:** Преди изтриването $A[1..i-1]$ е трансформирано в $B[1..j]$ оптимално. $|OPT| = dp[i-1][j] + 1$.
- **Вмъкване на $b[j]$:** Преди вмъкването $A[1..i]$ е трансформирано в $B[1..j-1]$ оптимално. $|OPT| = dp[i][j-1] + 1$.

Тъй като трите случая са изчерпателни, $|OPT|$ е равно на минимума от трите — точно нашата формула.

Пример: “kitten” $\rightarrow$ “sitting”

$A = \text{kitten}$ ($m = 6$), $B = \text{sitting}$ ($n = 7$).

	ε	s	i	t	t	i	n	g
ε	0	1	2	3	4	5	6	7
k	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2	3	4	5
t	4	4	3	2	1	2	3	4
e	5	5	4	3	2	2	3	4
n	6	6	5	4	3	3	2	3

Отговор: $dp[6][7] = 3$. Трите операции са: замяна $k \rightarrow s$, замяна $e \rightarrow i$, вмъкване на g .

Обобщение

Задача	Подзадача	Сложност
0/1 Раница	$dp[i][j]$ - макс. стойност с i предмета и капацитет j	$O(n \cdot W)$
Верижно умножение	$dp[i][j]$ - мин. умножения за $A_i \cdots A_j$	$O(n^3)$
Edit Distance	$dp[i][j]$ - мин. операции за $A[1..i] \rightarrow B[1..j]$	$O(m \cdot n)$

Ключовият урок: при всяка задача намираме *последното взето решение* в оптималния отговор, разглеждаме всички случаи за него и вземаме оптималния. Подзадачите трябва да са строго по-малки - гаранция за коректност и отсъствие на цикли.